Imam Mohammad Ibn Saud Islamic University

College of Computer and Information Sciences

Computer Science Department

Course Title:	Design and Analysis of Algorithms
Course Code:	CS1211
Course Instructor:	Dr. Abdulrahman Albarrak
Project:	Maximize Cash Back Rewards
Semester:	Third Semester, 2024/2025
Due Date:	Sunday, May 18, 2025 (20-11- 1446)
Marks:	20

Names	Academic Number	Section
Khalid Abdullah Al Dosari	**443018297**	**172**
Abdulellah Abdulrahman Altowaijri	**444003866**	
Zeyad Moawad Nazir	**444000368**	

0.0 Table of Contents

1.0 Project Introduction.....3

1.1 Overview:.....3

1.2 Brute Force Method:.....3

1.3 Characteristics of Brute Force Method:.....3

1.4 Greedy Algorithm:.....3

1.5 Characteristics of Greedy Method:.....3

1.6 Similarities:.....4

1.7 Key Differences.....4

1.7.1 Solution Quality:.....4

1.7.2 Computational Resources:.....4

1.7.3 Approach:.....4

1.7.4 Scalability:.....4

1.8 Conclusion:.....4

2.0 Pseudocode.....5

2.1 Brute-Force Algorithm.....5

2.2 Greedy algorithm.....6

3.0 Time Complexity Analysis.....7

3.1 Brute Force Algorithm.....7

3.2 Greedy Algorithm.....7

3.3 Comparison:.....8

3.4 Theoretical examples.....8

3.5 Runtime examples:.....9

4.0 Used Equations.....10

1.0 Project Introduction

1.1 Overview:

In this project, there are two completely different algorithm schemes that will attempt to solve the cashback optimization problem: brute force and greedy. Both will aim to maximize cash back rewards while respecting the spending limits of the cards, but they have very different schemes and natures.

1.2 Brute Force Method:

The brute force approach is a systematic method that:

- Examines all possible combinations of card-category assignments
- For n cards and m categories, it generates n^m permutations
- Evaluates each permutation to ensure it respects card spending limits
- Calculates the total cashback for each valid permutation
- Selects the permutation that yields the maximum cashback

1.3 Characteristics of Brute Force Method:

- **Completeness:** Guarantees finding the optimal solution
- **Time Complexity:** O(n^m), making it computationally expensive
- **Space Complexity:** Requires storing all possible permutations
- **Best Use Case:** When the number of cards and categories is small

1.4 Greedy Algorithm:

The greedy approach is a heuristic method that:

- Makes locally optimal choices at each step
- Assigns each category to the card offering the highest cashback rate
- Only considers assignments that don't exceed card limits
- Builds the solution incrementally, one category at a time

1.5 Characteristics of Greedy Method:

- **Efficiency:** Much faster than brute force
- **Time Complexity:** O(nm), where n is number of cards and m is number of categories
- **Space Complexity:** Minimal, only needs to track current assignments
- **Best Use Case:** When quick solutions are needed for large problems

1.6 Similarities:

- Objective: Both methods aim to maximize total cashback
- Constraints: Both respect card spending limits
- Input Requirements: Both use the same input data (spending amounts, rates, and limits)
- Output Format: Both provide detailed assignment of categories to cards

1.7 Key Differences

1.7.1 Solution Quality:

- Brute Force: Guarantees optimal solution
- Greedy: May not always find the optimal solution

1.7.2 Computational Resources:

- Brute Force: High computational cost
- Greedy: Low computational cost

1.7.3 Approach:

- Brute Force: Systematic exploration of all possibilities
- Greedy: Makes immediate best choices without considering future implications

1.7.4 Scalability:

- **Brute Force:** Limited by problem size
- **Greedy:** Can handle larger problems efficiently

1.8 Conclusion:

In this project, there are two completely different algorithmic schemes that will attempt to solve the cashback optimization problem: brute force and greedy. Both will aim to maximize cash back rewards while respecting the spending limits of the cards, but they have very different schemes and natures.

2.0 Pseudocode

2.1 Brute-Force Algorithm

```
Algorithm BruteForceCashback
Input:
    numberOfCards, numberOfCategories
    amountSpent[] // Array of spending amounts for each category
    rates[][] // 2D array of cashback rates (cards × categories)
    limit[] // Array of spending limits for each card
Output:
    maximumPermutation // Best assignment found
    maximumCashback // Maximum cashback amount

1. // Initialize variables
   base = numberOfCards
   numberOfPermutations = base^numberOfCategories
   allPermutations = empty list
   maximumCashback = -1
   maximumPermutation = null

2. // Generate all possible permutations
   FOR i = 0 TO numberOfPermutations-1
       temp = i
       currentPermutation = empty list
       FOR j = 0 TO numberOfCategories-1
           // Convert to base-numberOfCards and add 1
           currentPermutation.add(0, (temp % base) + 1)
           temp = temp / base
       END FOR
       allPermutations.add(currentPermutation)
   END FOR

3. // Evaluate each permutation
   FOR each permutation IN allPermutations
       IF checkPermutation(permutation) THEN
           cashback = calculateCashback(permutation)
           IF cashback > maximumCashback THEN
               maximumCashback = cashback
               maximumPermutation = permutation
           END IF
       END IF
   END FOR

4. RETURN maximumPermutation, maximumCashback

Function checkPermutation(permutation)
    // Reset spending tracker
    FOR i = 0 TO numberOfCards-1
        cardsTotalSpending[i] = 0
    END FOR

    // Calculate total spending for each card
    FOR i = 0 TO numberOfCategories-1
        cardIndex = permutation[i] - 1
        cardsTotalSpending[cardIndex] += amountSpent[i]
    END FOR

    // Check if any card exceeds its limit
    FOR i = 0 TO numberOfCards-1
        IF cardsTotalSpending[i] > limit[i] THEN
            RETURN false
        END IF
    END FOR
    RETURN true

Function calculateCashback(permutation)
    totalCashback = 0
    FOR i = 0 TO numberOfCategories-1
        cardIndex = permutation[i] - 1
        totalCashback += amountSpent[i] * (rates[cardIndex][i] / 100.0)
    END FOR
    RETURN totalCashback
```

2.2 Greedy algorithm

```
Algorithm GreedyCashback
Input:
    numberOfCards, numberOfCategories
    amountSpent[] // Array of spending amounts for each category
    rates[][] // 2D array of cashback rates (cards × categories)
    limit[] // Array of spending limits for each card
Output:
    chosen_cards[] // Array of card assignments
    totalCashback // Total cashback amount

1. // Initialize tracking arrays
   current_spent[] = array of zeros with size numberOfCards
   chosen_cards[] = array of size numberOfCategories
   totalCashback = 0

2. // Process each category
   FOR i = 0 TO numberOfCategories-1
       best_card = -1
       best_rate = -1

       // Find best available card
       FOR j = 0 TO numberOfCards-1
           IF current_spent[j] + amountSpent[i] ≤ limit[j] AND rates[j][i] > best_rate THEN
               best_rate = rates[j][i]
               best_card = j
           END IF
       END FOR

       // Check if valid card found
       IF best_card == -1 THEN
           RETURN "Failed to allocate category"
       END IF

       // Assign category to best card
       chosen_cards[i] = best_card
       current_spent[best_card] += amountSpent[i]
       totalCashback += amountSpent[i] * (rates[best_card][i] / 100.0)
   END FOR

3. RETURN chosen_cards, totalCashback
```

3.0 Time Complexity Analysis

3.1 Brute Force Algorithm

1. Number of cards: $\mathbf{n}$
2. Number of categories: $\mathbf{m}$
3. Number of permutations = $\mathbf{n^m}$
4. Time per permutation = $O(\mathbf{m})$
5. Total time for all permutations = $O(\mathbf{n^m \times m})$
6. Overall Time Complexity: $T(n, m) = O(\mathbf{n^m \times m})$

The brute force algorithm explores all possible card-to-category assignments by generating every permutation. For each permutation, it calculates the total cashback and checks its validity.

This results in **exponential complexity**, making the algorithm impractical for large datasets. While it guarantees the optimal solution, the performance is not scalable.

3.2 Greedy Algorithm

1. Number of cards: $\mathbf{n}$
2. Number of categories: $\mathbf{m}$
3. Outer loop (categories): $\mathbf{O(m)}$
4. Inner loop (cards): $\mathbf{O(n)}$
5. Overall Time Complexity: $\mathbf{T(n, m) = O(m \times n)}$

The greedy algorithm iteratively assigns the best available card for each spending category based on predefined criteria.

This approach offers polynomial complexity, making it highly efficient for larger inputs. However, it does not always guarantee the optimal solution, as it makes decisions locally for each category.

3.3 Comparison:

The brute force algorithm results in **exponential complexity**, making the algorithm impractical for large datasets. While it guarantees the optimal solution, the performance is not scalable.

The greedy algorithm on the other hand offers polynomial complexity, making it highly efficient for larger inputs. However, it does not always guarantee the optimal solution, as it makes decisions locally for each category.

3.4 Theoretical examples

Brute Force examples:

- 4 cards, 6 categories = 4^6 = 4096 permutations
  → **T(4,6) = O(4^6 * 6) = O(4096 * 6) = O(24,576)**
- 5 cards, 8 categories = 5^8 = 390,625 permutations
  → **T(4,6) = O(6 * 4) = O(24)**

Greedy Method examples:

- 4 cards, 6 categories = 24 operations
- 5 cards, 8 categories = 40 operations

This shows why the greedy algorithm is much more efficient for larger inputs, though it may not always find the optimal solution like the brute force approach does.

3.5 Runtime examples:

Example 1 input/Execution Time (Small Numbers):

```
Enter the number of cards:
4
Enter the number of categories:
6

Enter amount spent for each category:
Category 1: 600
Category 2: 400
Category 3: 200
Category 4: 300
Category 5: 800
Category 6: 500

Enter spending limit for each card:
Card 1: 1500
Card 2: 700
Card 3: 2500
Card 4: 3500
```

```
Running Time Analysis:
Brute Force Algorithm: 15.13 ms
Greedy Algorithm: 13.75 ms
Speedup Ratio (Brute Force/Greedy): 1.10

Theoretical Time Complexity:
Brute Force: O(n^m * m) where n=4 cards and m=6 categories
Greedy: O(m * n) where n=4 cards and m=6 categories
```

Example 2 input/Execution Time (Large Numbers):

```
Enter the number of cards:
6
Enter the number of categories:
8

Enter amount spent for each category:
Category 1: 600
Category 2: 400
Category 3: 200
Category 4: 300
Category 5: 500
Category 6: 800
Category 7: 1000
Category 8: 1500

Enter spending limit for each card:
Card 1: 1500
Card 2: 700
Card 3: 2500
Card 4: 3500
Card 5: 6500
Card 6: 10000
```

```
Running Time Analysis:
Brute Force Algorithm: 623.76 ms
Greedy Algorithm: 17.35 ms
Speedup Ratio (Brute Force/Greedy): 35.94

Theoretical Time Complexity:
Brute Force: O(n^m * m) where n=6 cards and m=8 categories
Greedy: O(m * n) where n=6 cards and m=8 categories
```

Conclusion: As we stated before, the Brute Force method's efficiency drops as numbers grow larger while the Greedy method barely changes.

4.0 Used Equations

Permutation generation

- Number of Permutations: numberOfPermutations = base^numberOfCategories
- Base Value: base = numberOfCards
- Digit Conversion: digit = (temp % base) + 1
- Next Digit: temp = temp / base

Spending calculations

- Card Total Spending: cardsTotalSpending[cardIndex] += amountSpent[i]
- Card Index: cardIndex = currentPermutation.get(i) - 1
- Limit Validation: cardsTotalSpending[i] > limit[i]

Cashback calculations

- Individual Category: cashback = amountSpent[i] * (rates[cardIndex][i] / 100.0)
- Total Cashback: totalCashback += amountSpent[i] * (rates[cardIndex][i] / 100.0)

Greedy algorithm equations

- Capacity Check: current_spent[j] + amountSpent[i] ≤ limit[j]
- Rate Comparison: rates[j][i] > best_rate
- Spending Update: current_spent[best_card] += amountSpent[i]
- Cashback Update: totalCashback += amountSpent[i] * (rates[best_card][i] / 100.0)

Random rate generation

- Rate Calculation: rates[i][j] = 1.0 + (5.0 * random.nextDouble())

Space complexity

- Brute Force: S(n,m) = O(n^m * m)
- Greedy: S(n,m) = O(n + m)

Validation equations

- Card Limit: cardsTotalSpending[i] > limit[i]
- Rate Check: rates[j][i] > best_rate
- Capacity Check: current_spent[j] + amountSpent[i] ≤ limit[j]

Index conversions

- Card Index: cardIndex = currentPermutation.get(i) - 1
- Rate Access: rates[cardIndex][i]
- Amount Access: amountSpent[i]
- Limit Access: limit[i]